



ЗВІТ

Лабораторна робота № 1

Дисципліна: Базы даних та інформаційні системи

**Тема: Ознайомлення з системами керування базами даних (СКБД) на
прикладі PostgreSQL.**

Виконав
Студент 3 курсу
Групи МІТ-31
Факультету інформаційних технологій
Бабяк Володимир

Мета: Ознайомитися із базовими можливостями PostgreSQL як реляційної системи керування базами даних, закріплення теоретичних знань із основ реляційної моделі даних, створення та модифікація баз даних, а також виконання базових операцій SQL.

Хід роботи

ВАРІАНТ № 4

1. Короткий опис бази даних База даних cinema_db розроблена для управління процесами кінотеатру. Вона містить три взаємопов'язані таблиці:

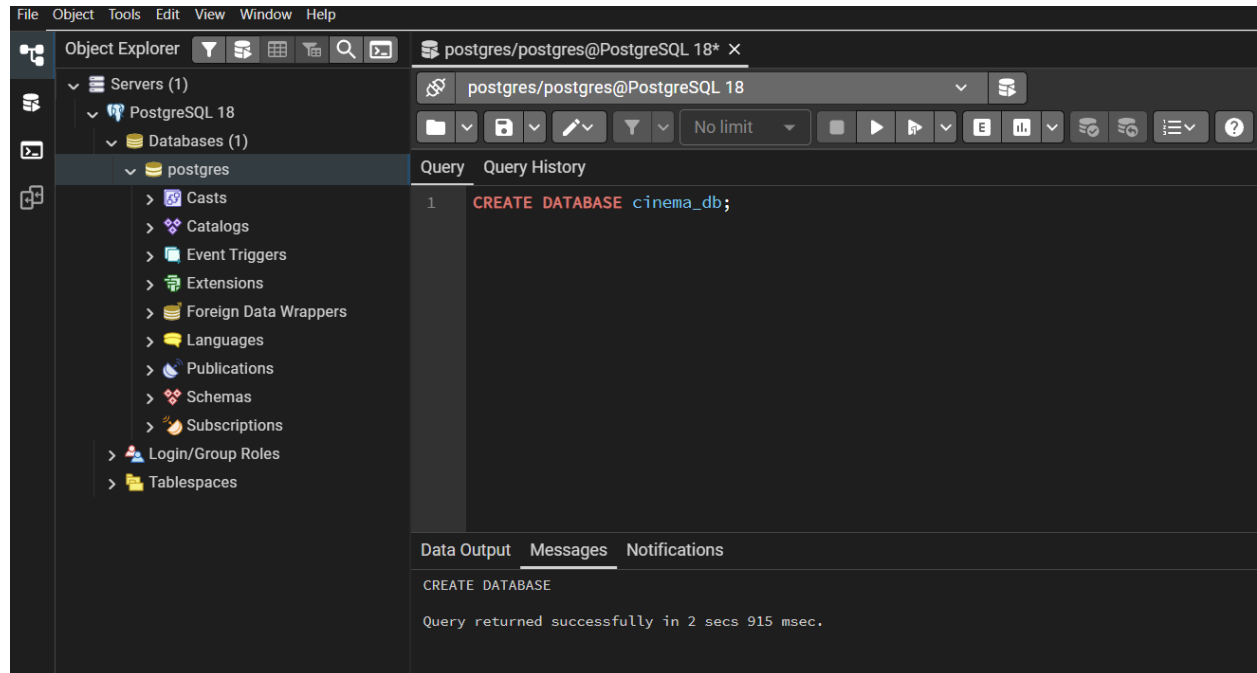
movies: Зберігає інформацію про фільми (ідентифікатор, назва, жанр, тривалість). Первинний ключ movie_id.

sessions: Містить розклад сеансів (ідентифікатор, час початку, номер залу). Первинний ключ session_id. Має зовнішній ключ movie_id для зв'язку з фільмами.

tickets: Фіксує продані квитки (ідентифікатор, номер місця, ціна). Первинний ключ ticket_id. Містить зовнішній ключ session_id, який створює зв'язок із відповідним сеансом.

2. Створення бази даних та таблиць Було виконано запуск сервера PostgreSQL та створено базу даних cinema_db.

```
CREATE DATABASE cinema_db;
```



Створено структуру таблиць та налаштовано зв'язки (первинні та зовнішні ключі):

CREATE TABLE movies (

movie_id SERIAL PRIMARY KEY,

title VARCHAR(150) NOT NULL,

genre VARCHAR(50),

duration_minutes INT

);

CREATE TABLE sessions (

session_id SERIAL PRIMARY KEY,

movie_id INT NOT NULL,

session_time TIMESTAMP,

hall_number INT,

FOREIGN KEY (movie_id) REFERENCES movies(movie_id) ON DELETE CASCADE

);

CREATE TABLE tickets (

ticket_id SERIAL PRIMARY KEY,

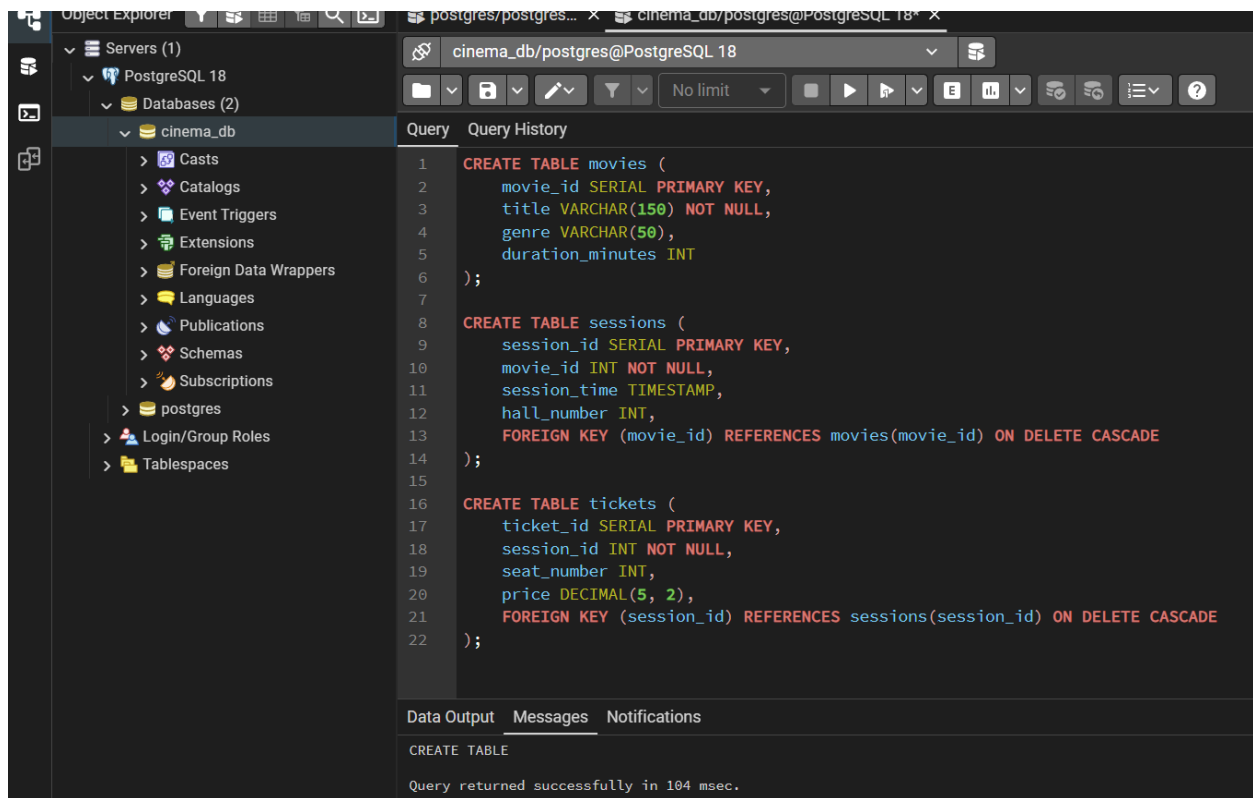
session_id INT NOT NULL,

seat_number INT,

price DECIMAL(5, 2),

FOREIGN KEY (session_id) REFERENCES sessions(session_id) ON DELETE CASCADE

);



3. Виконання базових операцій SQL (CRUD) Дані було додано в таблиці за допомогою оператора INSERT:

INSERT INTO movies (title, genre, duration_minutes) VALUES

('Дюна: Частина друга', 'Фантастика', 166),

('Дедпул і Росوماха', 'Бойовик', 127);

INSERT INTO sessions (movie_id, session_time, hall_number) VALUES

(1, '2026-05-10 18:00:00', 1),

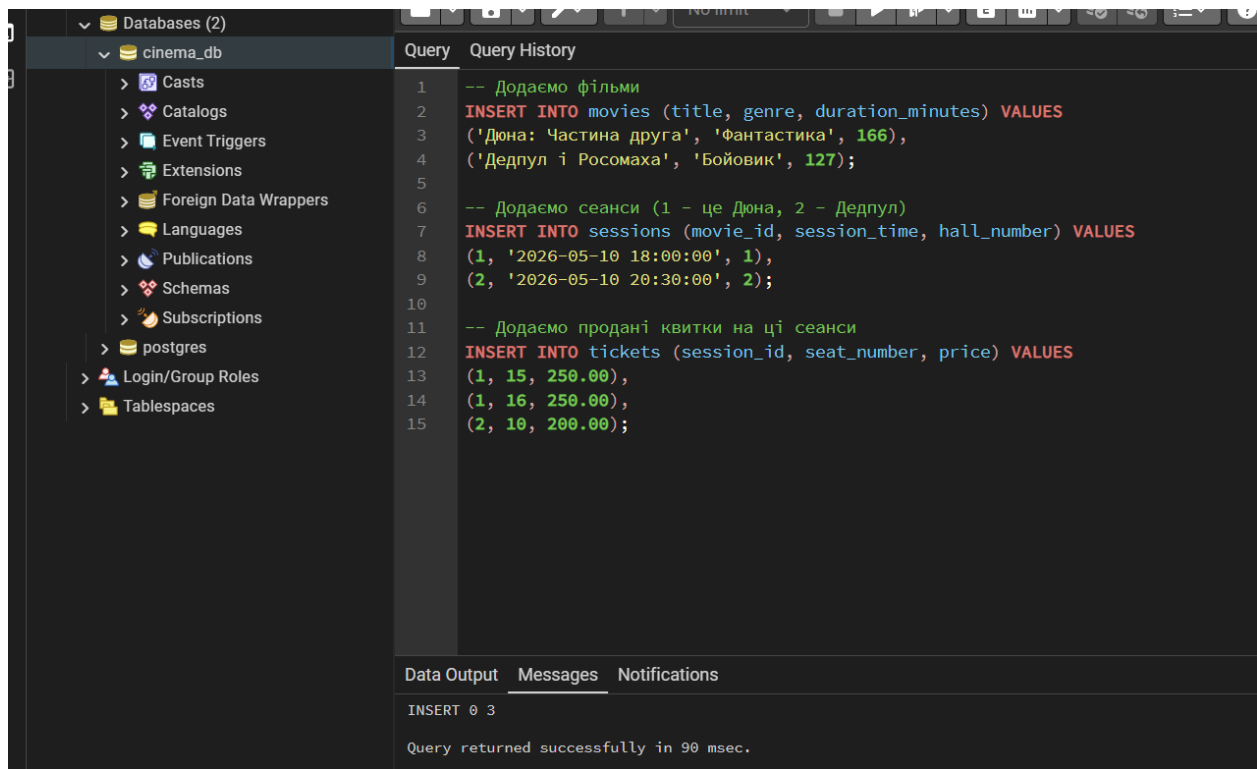
(2, '2026-05-10 20:30:00', 2);

INSERT INTO tickets (session_id, seat_number, price) VALUES

(1, 15, 250.00),

(1, 16, 250.00),

(2, 10, 200.00);



The screenshot shows a PostgreSQL database interface with a sidebar on the left displaying the database structure, including 'cinema_db' and 'postgres'. The main area is divided into 'Query' and 'Query History' tabs. The 'Query' tab contains the following SQL code:

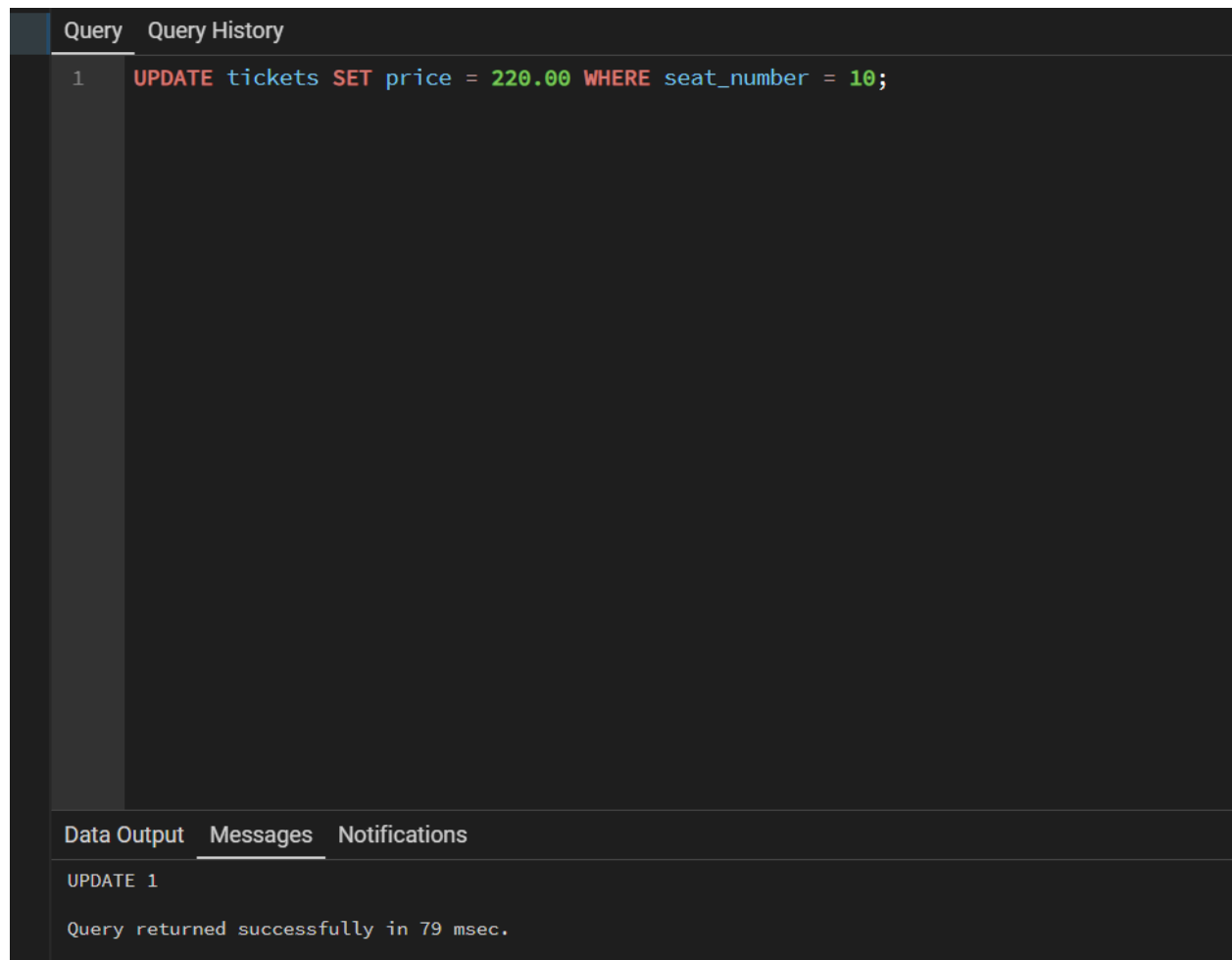
```
1 -- Додаємо фільми
2 INSERT INTO movies (title, genre, duration_minutes) VALUES
3 ('Дюна: Частина друга', 'Фантастика', 166),
4 ('Дедпул і Росوماха', 'Бойовик', 127);
5
6 -- Додаємо сеанси (1 - це Дюна, 2 - Дедпул)
7 INSERT INTO sessions (movie_id, session_time, hall_number) VALUES
8 (1, '2026-05-10 18:00:00', 1),
9 (2, '2026-05-10 20:30:00', 2);
10
11 -- Додаємо продані квитки на ці сеанси
12 INSERT INTO tickets (session_id, seat_number, price) VALUES
13 (1, 15, 250.00),
14 (1, 16, 250.00),
15 (2, 10, 200.00);
```

Below the query editor, the 'Data Output' tab is active, showing the result of the INSERT queries:

```
INSERT 0 3
Query returned successfully in 90 msec.
```

Ціну квитка змінено оператором UPDATE:

UPDATE tickets SET price = 220.00 WHERE seat_number = 10;

A screenshot of a SQL IDE interface. The top bar has two tabs: 'Query' and 'Query History'. The 'Query' tab is active, showing a single query: 'UPDATE tickets SET price = 220.00 WHERE seat_number = 10;'. The query is color-coded: 'UPDATE' is red, 'tickets' is blue, 'SET' is red, 'price' is blue, '=' is black, '220.00' is green, 'WHERE' is red, 'seat_number' is blue, '=' is black, and '10' is green. Below the query editor, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the text 'UPDATE 1' and 'Query returned successfully in 79 msec.'.

```
Query  Query History
1      UPDATE tickets SET price = 220.00 WHERE seat_number = 10;

Data Output  Messages  Notifications
UPDATE 1
Query returned successfully in 79 msec.
```

Для перегляду проданих квитків з інформацією про фільм та час сеансу виконано запит SELECT з використанням об'єднання JOIN:

SELECT

m.title AS movie_name,

s.session_time,

t.seat_number,

t.price

FROM tickets t

JOIN sessions s ON t.session_id = s.session_id

JOIN movies m ON s.movie_id = m.movie_id;

Query Query History

```
1 SELECT
2     m.title AS movie_name,
3     s.session_time,
4     t.seat_number,
5     t.price
6 FROM tickets t
7 JOIN sessions s ON t.session_id = s.session_id
8 JOIN movies m ON s.movie_id = m.movie_id;
```

Data Output Messages Notifications

Showing rows: 1 to 3

	movie_name character varying (150)	session_time timestamp without time zone	seat_number integer	price numeric (5,2)
1	Дюна: Частина друга	2026-05-10 18:00:00	15	250.00
2	Дюна: Частина друга	2026-05-10 18:00:00	16	250.00
3	Дедпул і Росомаха	2026-05-10 20:30:00	10	220.00

Тестове видалення запису з бази даних виконано за допомогою DELETE:

DELETE FROM tickets WHERE ticket_id = 1;

Query Query History

```
1 DELETE FROM tickets WHERE ticket_id = 1;
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 116 msec.

4. Посилання на репозиторій GitHub із виконаною роботою

<https://github.com/volodimirbabak/databases-course>

Висновок

Під час виконання лабораторної роботи я успішно встановив та налаштував СКБД PostgreSQL. Згідно з варіантом було розроблено базу даних cinema_db та створено три реляційні таблиці із встановленням зв'язків через первинні та зовнішні ключі. Практично закріплено навички написання основних SQL-запитів для керування даними, а саме операції вставки, оновлення, вибірки (включаючи JOIN) та видалення інформації. Результати роботи успішно оформлено та збережено у віддаленому репозиторії GitHub.